

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

Best Case = $O(1)$

Karena Target ditemukan di indeks pertama ($i=0$)

Average Case = $O(n/2) = O(n)$ Target berada di tengah-tengah array

Worst Case = $O(n)$ Target di akhir array atau tidak ada

Analisis: Algoritma memeriksa elemen satu per satu. Jika array berisi n elemen, paling buruk harus memeriksa semua n elemen $\rightarrow O(n)$.

```
def binary_search(arr, target):  
    left, right = 0, len(arr) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1
```

Best Case = $O(1)$

Target tepat di posisi mid pertama

Average Case = $O(\log n)$

Setiap iterasi membuang setengah array

Worst Case = $O(\log n)$

Target tidak ada / di ujung, pencarian terus dibagi dua

Analisis: Setiap iterasi, ruang pencarian dibagi 2. Untuk array n elemen, maksimal dilakukan $\log_2(n)$ langkah $\rightarrow O(\log n)$.

```
def binary_search_recursive(arr, left, right, target):  
    if left > right:  
        return -1  
    mid = (left + right) // 2  
    if arr[mid] == target:  
        return mid  
    elif arr[mid] < target:  
        return binary_search_recursive(arr, mid + 1, right, target)  
    else:  
        return binary_search_recursive(arr, left, mid - 1, target)
```

Analisis: Logika sama dengan iteratif, namun menggunakan call stack rekursi. Jumlah pemanggilan rekursif = kedalaman pohon rekursi = $\log_2(n) \rightarrow O(\log n)$. Space complexity: $O(\log n)$ karena stack rekursi.

Best Case = $O(1)$

Target ditemukan saat pemanggilan rekursi pertama

Average Case = $O(\log n)$

Sama seperti versi iteratif

Worst Case = $O(\log n)$

Rekursi terjadi sebanyak $\log_2(n)$ kali

```
def jump_search(arr, target):  
    n = len(arr)
```

```

step = int(math.sqrt(n))
prev = 0
while arr[min(step, n) - 1] < target:
    prev = step
    step += int(math.sqrt(n))
    if prev >= n: return -1
for i in range(prev, min(step, n)):
    if arr[i] == target: return i
return -1

```

Analisis: Array dibagi menjadi blok-blok berukuran $\sqrt{n}$. Fasa lompatan: $O(\sqrt{n})$, fasa linear search dalam blok: $O(\sqrt{n})$. Total $\rightarrow O(\sqrt{n})$

Best Case = $O(1)$

Target ada di blok pertama, elemen pertama

Average Case = $O(\sqrt{n})$

Butuh beberapa lompatan + linear search kecil

Worst Case = $O(\sqrt{n})$

Lompatan: $\sqrt{n}$ langkah + linear search dalam blok: $\sqrt{n}$ langkah

```

def interpolation_search(arr, target):
    ...
    pos = low + ((target - arr[low]) * (high - low) // (arr[high] - arr[low]))
    ...

```

Analisis: Menggunakan rumus interpolasi untuk memperkirakan posisi target secara proporsional. Jauh lebih efisien dari binary search bila data seragam, tapi bisa jelek bila distribusi data miring/skewed

Best Case = $O(1)$

Target langsung ditemukan dari formula interpolasi

Average Case = $O(\log \log n)$

Data terdistribusi seragam (uniform)

Worst Case = $O(n)$

Data terdistribusi tidak seragam